Dive Into Pentesting

Pentesting VS Malicious Hacking

Penetration testing, or "pentesting", is an authorised security assessment that is performed on systems, applications, or networks. It is carried out with explicit permission from the system owner within agreed boundaries.

Penetration Testing vs Malicious Hacking

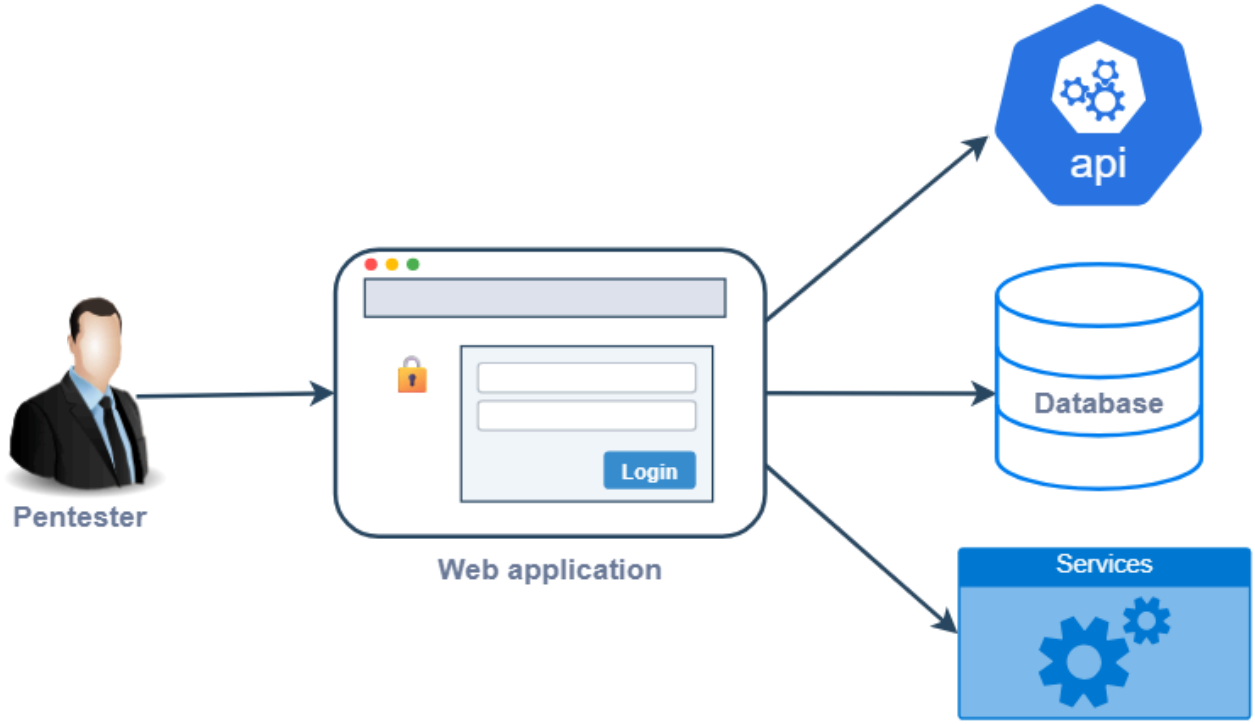
Core Factor	Penetration Tester	Malicious Attacker
Authorisation	Penetration testers operate with explicit written consent from the system owner.	Attackers operate without consent and violate the rights and security of the organisation.
Scope	Penetration testers operate within a clearly defined scope that is set by the organisation in order to protect critical systems.	Attackers are not restricted by scope and will target anything that helps them achieve their goal.
Coverage	Penetration testers aim for broad coverage and assess multiple areas of a system.	Attackers focus on the quickest or most effective path to success, and often target systems and data that can provide financial benefit.
Responsibility	Penetration testers are accountable for their actions and the impact of their work, and are expected to act professionally at all times.	Attackers do not feel accountable for their actions and do not take responsibility for the damage that they cause.

Penetration Testing Focus Areas

To achieve meaningful coverage, it is important to understand the different areas that a penetration tester looks at

Web Application Penetration Testing

focuses on finding gaps and weaknesses in a web application. Testing is commonly performed from a user perspective by interacting with the application's user interface and its APIs. The goal is to assess how the application handles user input, authentication, authorisation, sessions, and data processing. Weaknesses in web applications can have great impact on its users because web applications are usually exposed to the internet.



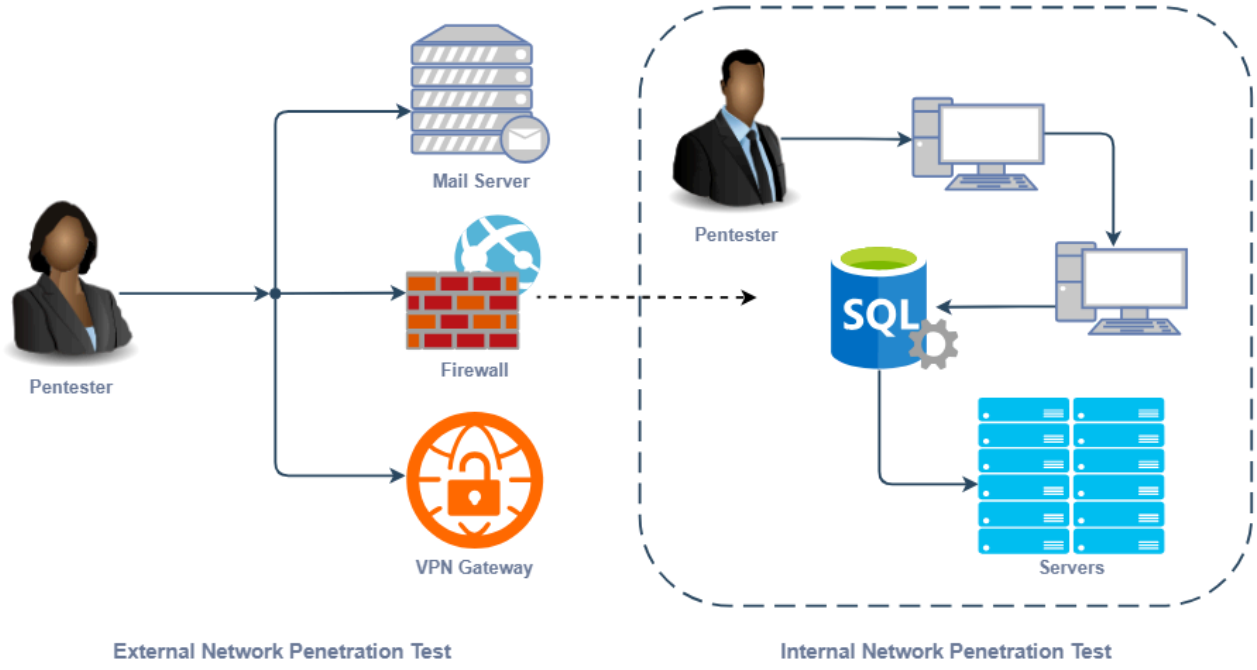
- **Authentication**: This area is evaluated for weaknesses in the application's credential handling, password policy, account lockout mechanisms, multi-factor authentication (MFA) implementation, protection against automated attacks such as brute-force and credential stuffing, password-reset flow and overall authentication-related logics.
- **Authorisation**: This area is evaluated for weaknesses in the application's access control mechanism, ensuring users can only access resources and perform actions permitted by their role, and protection against vertical and horizontal privilege escalation attacks.
- **Session management**: This area is evaluated for weaknesses in how sessions are created, maintained, and invalidated. This includes session fixation risks, session invalidation after logout, idle timeout enforcement, secure cookie attributes, and protection against cross-site request forgery (CSRF).
- **Input and output validation**: This area is evaluated for weaknesses in the application's data handling controls, including protection against injection attacks, data type validation, and output handling.
- **Security configuration**: This area is evaluated for gaps in the server and application configurations, including security headers, error handling behaviour, rate-limiting controls, cryptographic configuration, and exposure of unnecessary services or features.

Network Penetration Testing

Network penetration testing focuses on finding vulnerabilities in the underlying infrastructure that connects systems together. Testing could be performed from an external or an internal user perspective.

External network penetration testing is performed from an external user perspective with little to no access to information about the externally exposed systems. This type of assessment focuses on external-facing infrastructure such as internet-facing servers, firewalls, VPN gateways, and remote access services.

Internal network penetration testing, on the other hand, is an "assumed breach" scenario where a threat actor already has access to a system in the network. This type of assessment evaluates what an attacker could do next, such as moving between systems, escalating privileges, or accessing sensitive data.



- **Authentication mechanisms**: This area is evaluated for weaknesses in network-level authentication controls such as password policy, multi-factor authentication (MFA) enforcement, credential reuse, use of default credentials, and protection against attacks on services such as admin portals, VPN, SSH, or RDP.
- **Authorisation and access controls**: This area is evaluated for weaknesses in network access control mechanisms, ensuring users and systems can only access resources permitted by their role or trust level.
- **Network segmentation and trust relationships**: This area is evaluated for weaknesses in user and system trust relationships, firewall rules, and isolation controls.
- **Configuration and patch management**: This area is evaluated for gaps in device configurations and services, such as use of outdated software, insecure default configuration, and weak encryption protocols.

Vulnerability, Threat and Risk

Vulnerability

A vulnerability is a weakness or gap in an organisation's environment that could be exploited to compromise the security of systems, data, or operations.

weaknesses in software, systems, or configurations that can be exploited due to coding errors, insecure settings, or flawed system design.

example, a web server running outdated software with known flaws is a vulnerability. The weakness exists, but nothing happens unless it is taken advantage of.

Threat

A threat is anything that can exploit a vulnerability and cause harm to an organisation's environment. It represents a source of danger that can take advantage of a weakness to compromise the confidentiality, integrity, and availability of the organisation's systems.

Threats may include malicious actors such as cybercriminals, insider threats, or tools that are used to perform automated attacks.

They may also be non-malicious events like system failures, human error, or environmental incidents.

For example, an attacker scanning the internet for outdated servers is a threat. They have the capability and intent to exploit the vulnerability.

Risk

Risk is the potential damage that could occur if a threat successfully exploits a vulnerability

We can determine the overall risk of a vulnerability by combining the impact and likelihood of exploitation

The following formula is commonly used to simplify the calculation of overall risk: `Vulnerability * Threat = Risk`.

Low Risk

Imagine a web application displays detailed error messages when invalid input is submitted.

- **Vulnerability:** Verbose error messages revealing internal file paths
- **Threat:** Attackers triggering errors to gather system information
- **Risk:** Limited information disclosure that may help reconnaissance but does not directly compromise the system

High Risk

Imagine a web application that allows users to change an account ID parameter in a request and access other users' data.

- **Vulnerability:** Broken access control allowing unauthorised access to user data
- **Threat:** Attackers modifying request parameters to retrieve or alter other sensitive user information
- **Risk:** Exposure or manipulation of sensitive customer data, leading to data privacy violations and regulatory consequences

Risk Management

Risk management is a structured process that many organisations use to identify, evaluate, and control security risks over time. Effective risk management means understanding which risks matter most and applying appropriate controls to eliminate or reduce their impact or likelihood of exploitation.

Risk management is typically an ongoing cycle consisting of four stages:

- **Identification**: Identifying assets, vulnerabilities, and potential threats that could affect the organisation.
- **Analysis**: Evaluating risks to determine the potential impact and likelihood of exploitation, which translates to the severity of each risk and helps plan remediation efforts.
- **Mitigation**: Reducing or controlling identified risks. This may include applying security patches, strengthening access controls, improving configurations, implementing monitoring tools, or redesigning insecure processes.
- **Monitoring**: Continuous monitoring of risks to ensure that controls remain effective, new vulnerabilities are detected, and emerging threats are addressed promptly.

In some cases, organisations may choose to accept or transfer risk if mitigation is not practical or cost-effective. Accepting a risk is usually chosen if the impact is minimal and the cost of reducing the risk outweighs the benefit. Transferring risk means shifting the responsibility to a third party, such as purchasing an insurance policy that would cover the cost that comes with the risk.

Why Vulnerabilities Exist

Vulnerabilities can occur through flaws, features, or human error, which can result in compromise of data and resources, and attackers can exploit one or more to achieve their goal.

Human Assumptions

Developers who are not security-aware often assume that users will use systems as intended. Attackers intentionally challenge this assumption by using systems in unintended ways to

expose weaknesses.

For instance, a developer who assumes that users will only upload image files as their profile picture. The lack of file validation controls allows an attacker to upload a malicious script, which is then executed by the server.

Software Bugs

Programming errors can introduce unintended behaviour that could pose a security risk, which may arise from logic mistakes and incomplete validation. A poorly written code may work as designed, but could be flawed if not securely coded.

For instance, a web application fails to properly validate input when processing form data that interacts with a database. An attacker submits a specially crafted input that alters the SQL query, enabling them to gain unauthorised access to sensitive data.

System Complexity

Modern environments consist of many interconnected components such as APIs, microservices, databases, and third-party integrations. The likelihood of a misconfiguration occurring in these complex environments is higher, which could lead to an exploitable vulnerability.

For instance, a company with multiple services integrated into their web application, like an authentication provider or third-party payment processor, might overlook a misconfiguration that exposes administrative APIs. Due to the complexity of interactions between these components, this misconfiguration allows an attacker to access critical administrative APIs that were not meant to be exposed.

Over Customisation

Extensive customisation of software or workflows can introduce inconsistencies and unintended security gaps. Custom features may not follow standard security practices, and heavily modified systems can be difficult to maintain, update, or patch properly.

For instance, an organisation that develops a custom authentication feature instead of using a standard login framework may implement their own password storage, session handling, and account recovery logic in order to meet their internal requirements. However, these can be difficult to maintain over time. Customisations can become outdated and lead to insecure practices like weak hashing algorithms or inconsistent session timeouts.

Technical and Design Flaws

Technical and design flaws could occur when security is not built into the design from the beginning. The source of these flaws may not be intentional, but can pose a huge risk in an organisation's security.

For instance, implementing multi-factor authentication in an existing web application. Developers might implement the feature, but overlook the existing design where a fully authenticated session cookie is issued before the MFA process is completed. This oversight allows an attacker to navigate to authenticated pages without completing the MFA process.

Common Causes and Resulting Vulnerabilities

Root Cause	Example Scenario	Resulting Vulnerability
Human Assumptions	Developer assumes users only upload images to a profile photo field, no validation is in place.	Unrestricted file upload: An attacker uploads a web shell and executes OS commands on the server.
Software Bugs	Form data is concatenated into a database query rather than handled through parameterised inputs.	SQL injection: An attacker crafts input to extract or modify sensitive records from the database.
System Complexity	Multiple integrated services make it easy to overlook a misconfigured API endpoint.	Exposed admin API: An attacker reaches admin functions that were never meant to be publicly accessible.
Over-Customisation	Custom authentication replaces a standard framework with logic that is hard to maintain.	Weak authentication: Outdated hashing and broken session logic expose user accounts to takeover.
Technical and Design Flaws	Authenticated session is issued before the MFA step is completed.	MFA bypass: An attacker navigates to authenticated pages without completing the MFA process.

The Penetration Tester Mindset

Technical skills alone are not enough to define the proficiency of a penetration tester. While understanding how vulnerabilities can be exploited is important, knowing how to effectively execute penetration tests is equally valuable.

Effective Mindset	Ineffective Mindset
Understand the system: Study how the target operates before testing	Rushing to exploitation: Attacking without understanding the system
Attention to detail: Notice subtle behavioural differences	Ignoring context: Reporting findings without business relevance
Constant curiosity: Ask "what if?" to uncover new opportunities	Over-reliance on tools: Missing logic flaws that require manual testing
Prioritise critical areas: Focus effort on high-impact functions first	Making assumptions: Guessing system behaviour without verification
Think in context: Assess risk by real-world consequence	Tunnel vision: Fixating on one area at the cost of coverage
Creative thinking: Chain vulnerabilities for greater impact	Blindly following a checklist: Missing deeper issues outside the list

Common Best Practices During a Penetration Test

Maintaining Good Notes

Keeping detailed notes throughout a penetration test helps a tester track their activity, findings, and observations as the assessment progresses

Collecting Evidence

collecting evidence such as screenshots, tool output, or logs supports the validity of findings and provides clear proof of impact

Managing Time

Managing time effectively allows a tester to balance effort across different areas and ensure that sufficient time is spent on critical functions and reporting

Proactive Communication

helps keep the engagement on track by providing regular progress updates and highlighting blockers early

Staying Professional

Respecting scope boundaries, handling sensitive data responsibly, and conducting testing in a controlled manner are some of the essential habits that maintain professionalism.

Ethics. Permission and Trust

Ethics

- Respecting the defined scope and avoiding activities outside authorised boundaries
- Avoiding actions that could disrupt systems and services
- Handling sensitive data responsibly and only accessing data that is required to demonstrate impact
- Stopping and reporting unexpected access to highly sensitive or out-of-scope systems
- Redacting sensitive data in reports to prevent unintended disclosure

Permission

- Obtaining written authorisation before initiating any testing activities
- Defining a clear scope and adhering to the agreed-upon scope of testing
- Confirming testing windows and approved methods before conducting potentially disruptive actions
- Seeking clarification when uncertainty exists about whether a system or action is in scope
- Pausing and notifying the organisation if activities may exceed authorised boundaries

Trust

- Providing status updates periodically to reassure stakeholders that testing is progressing as planned
- Being transparent about limitations, blockers, and uncertainties during testing in order to receive proper assistance
- Reporting findings accurately and clearly demonstrating business impact
- Providing actionable recommendations that help eliminate or mitigate risks
- Addressing organisational concerns promptly and adapting communication style to technical and non-technical stakeholders